

Aluno: Matheus Cardoso Pereira Santos **Registro Acadêmico:** a2609380

Disciplina: Teste de Software

Professor: Bernardo Lima

Prática de Laboratório 02

(0,5) Elabore um fichamento do Capítulo 5: Teste de Mutação do livro Introdução ao Teste de Software, disponível no moodle. A ficha deve conter, no mínimo:

- a. Um resumo do texto, apresentando o contexto, os objetivos do capítulo, e seus conteúdos principais (deve conter em volta de 600 palavras)**
- b. Um conjunto de citações diretas indicando o conteúdo mais relevante do texto**
- c. Uma lista dos pontos principais de cada seção**

Resposta:

FICHAMENTO TEXTUAL – CAPÍTULO 5
DELAMARO, Márcio Eduardo; MALDONADO, José Carlos; JINO, Mario. Introdução ao Teste de Software . Rio de Janeiro: Elsevier, 2007.
O Capítulo 5 aborda o Teste de Mutação, também chamado de Análise de Mutantes, um critério de teste fundamentado na técnica baseada em defeitos. O ponto de partida é a premissa já discutida: o objetivo do teste não é provar que o programa está correto, mas revelar defeitos. Como o teste exaustivo é inviável, o Teste de Mutação propõe uma solução prática: injetar defeitos típicos no programa original, gerando versões alteradas (os mutantes), e forçar o testador a criar casos de teste capazes de diferenciar o programa original de suas versões defeituosas. Dois pilares teóricos sustentam o critério: (1) a hipótese do programador competente, segundo a qual um programa desenvolvido por profissionais competentes está próximo da versão correta; e (2) o efeito de acoplamento, que indica que casos de teste capazes de revelar defeitos simples também são eficazes na detecção de defeitos mais complexos. A aplicação prática do Teste de Mutação envolve quatro etapas: (a) geração dos mutantes a partir de operadores de mutação; (b) execução do programa original; (c) execução dos mutantes; (d) análise dos mutantes vivos (sobreviventes). Um mutante morre quando um caso de teste produz resultado diferente do programa original. Um mutante que sobrevive pode ser sintaticamente diferente, mas semanticamente equivalente (mutante equivalente), ou indicar que o conjunto de testes é insuficiente. A qualidade do conjunto de testes é medida pelo escore de mutação, calculado pela razão entre o número de mutantes mortos e o total de mutantes não equivalentes. Quanto maior o escore, maior a capacidade do teste de revelar defeitos. O capítulo reconhece a principal limitação da técnica: o alto custo computacional, devido ao grande número de mutantes gerados. Para mitigar esse problema, são apresentadas estratégias como mutação fraca (que reduz o escopo da execução), amostragem aleatória de mutantes e mutação seletiva/restrita. Ferramentas como Mothra e Proteum são citadas como suporte essencial.

Por fim, o capítulo menciona extensões da técnica, como a Mutação de Interface, focada em falhas de integração entre componentes, e a aplicação de mutação em modelos como Máquinas de Estados Finitos e Redes de Petri.

Citações diretas:

- "O Teste de Mutação ou Análise de Mutantes, como também é conhecido, é um critério de teste da técnica baseada em defeitos."
- "No caso do Teste de Mutação, o programa que está sendo testado é alterado diversas vezes, criando-se um conjunto de programas alternativos ou mutantes, como se defeitos estivessem sendo inseridos no programa original."
- "O trabalho do testador é escolher casos de teste que mostrem a diferença de comportamento entre o programa original e os programas mutantes."
- "A hipótese do programador competente afirma que todo programa criado por um programador competente está correto ou está 'próximo' do correto."
- "O escore de mutação varia entre 0 e 1 e fornece uma medida objetiva de quanto o conjunto de casos de teste analisado aproxima-se da adequação."

Pontos principais de cada seção:

- 1.1 **Introdução:** Contextualiza o objetivo dos testes, apresenta a técnica baseada em defeitos e introduz o conceito geral do Teste de Mutação e dos subdomínios de entrada.
- 1.2 **Histórico:** Revisa a origem da técnica a partir de 1978 com DeMillo, Lipton e Sayward, aborda a "mutação fraca" e resume a evolução da aceitação da técnica pela comunidade acadêmica.
- 1.3 **Definições e conceitos básicos:** Discute a inviabilidade da prova de correção universal e introduz a correção relativa usando o conceito de "vizinhança" de um programa.
- 1.4 **Aplicação do Teste de Mutação:** Detalha as quatro etapas fundamentais da técnica: geração de mutantes, execução original, execução de mutantes e análise. Apresenta o cálculo do escore de mutação.
- 1.5 **Ferramentas:** Discute a necessidade de automação (devido ao alto volume de processamento) e apresenta as ferramentas Mothra e a família Proteum.
- 1.6 **Mutação de Interface:** Estende o conceito de mutação para testar falhas na integração e comunicação (parâmetros, variáveis globais) entre diferentes unidades do software.
- 1.7 **Outros trabalhos:** Menciona a flexibilidade do teste de mutação em validar não só códigos, mas artefatos executáveis e modelos teóricos (ex: Redes de Petri).
- 1.8 **Considerações finais:** Reforça que, apesar do custo computacional e da complexidade aparente, a técnica simula testes práticos rigorosos altamente efetivos.

FICHAMENTO TEXTUAL – CAPÍTULO 04

DELAMARO, Márcio Eduardo; MALDONADO, José Carlos; JINO, Mario. **Introdução ao Teste de Software**. Rio de Janeiro: Elsevier, 2007.

Resumo do texto:

O Capítulo 4 aborda o teste estrutural, também conhecido como técnica de "caixa branca", que deriva os requisitos de teste a partir da análise da implementação interna e da lógica do código-fonte. O objetivo da técnica é forçar a execução sistemática dos componentes elementares do programa, exercitando caminhos lógicos, condições, laços e o fluxo de dados das variáveis.

O texto reconhece as limitações inerentes ao teste estrutural, como a incapacidade de detectar "caminhos ausentes" (funcionalidades omitidas não possuem código a ser testado) e o problema da correção coincidente (quando dados de entrada mascaram falhas). Apesar dessas barreiras teóricas, o teste estrutural é considerado vital para a manutenção e garantia de qualidade de software.

Os critérios estruturais são classificados em três famílias: complexidade, fluxo de controle e fluxo de dados.

- **Família da complexidade:** Destaca-se a Métrica de McCabe, que calcula caminhos linearmente independentes no grafo de fluxo de controle.
- **Família do fluxo de controle:** Inclui os critérios populares Todos-os-Nós, Todas-as-Arestas e Todos-os-Caminhos.
- **Família do fluxo de dados:** Considerada mais avançada, foca na interação entre pontos de definição e uso de variáveis. São apresentados os critérios de Rapps e Weyuker e os critérios Potenciais-Usos de Maldonado, que resolvem deficiências dos testes de controle convencionais.

A fundamentação teórica utiliza o modelo formal de Grafo de Fluxo de Controle (GFC). Ferramentas de suporte, como a POKE-TOOL, são apontadas como essenciais para instrumentalizar o código e verificar a cobertura dos critérios.

Citações diretas:

- "A técnica estrutural (ou caixa branca) estabelece os requisitos de teste com base em uma dada implementação, requerendo a execução de partes ou de componentes elementares do programa"
- "Independentemente dessas desvantagens, a técnica estrutural é vista como complementar às demais técnicas de teste existentes, uma vez que cobre classes distintas de defeitos"
- "Os critérios baseados em fluxo de controle utilizam apenas características de controle da execução do programa, como comandos ou desvios, para determinar quais estruturas são necessárias."
- "As ocorrências de uma variável em um programa podem ser uma 'definição', uma 'indefinição' ou um 'uso'."
- "A aplicação de critérios de teste sem o apoio de ferramentas automatizadas tende a ser uma

atividade propensa a erros e limitada a programas muito simples."

Pontos principais de cada seção:

- **Introdução:** Define o teste caixa branca, expõe as suas bases e lista as limitações clássicas (indecebibilidade, caminhos não executáveis e ausentes).
- **Histórico:** Apresenta a evolução das famílias de critérios desde a Complexidade de McCabe e as hierarquias de fluxo de controle, até o surgimento das análises de fluxo de dados na década de 1970.
- **Definições e conceitos básicos:** Formaliza a técnica a partir da modelagem do Grafo de Fluxo de Controle (GFC), identificando nós, arcos, caminhos e definindo usos computacionais/predicativos de variáveis.
- **Crítérios de teste estrutural:** Detalha as três principais abordagens: Complexidade (McCabe), Controle (Nós, Arestas, Caminhos) e Dados (Rapps e Weyuker, e a família Potenciais-Usos).
- **Ferramentas:** Demonstra a inviabilidade de aplicar testes estruturais manualmente e apresenta soluções acadêmicas e comerciais, detalhando o funcionamento passo a passo da POKE-TOOL.
- **Considerações finais:** Reflete sobre a expansão dos critérios para o teste de integração (múltiplas unidades) e reforça a necessidade de estratégias de testes incrementais.

3. (0,5) Considerando o arquivo lab2.py ou a aplicação que vocês apresentaram nos seminários

a. Crie um conjunto de testes para cada uma das quatro funções com base em algum critério estrutural visto em aula.

b. Monte o GFC e calcule a complexidade ciclomática de pelo menos 4 funções

a. Conjunto de testes (Critério: Todas-Arestas / Todos os Desvios)

Para garantir a cobertura de todas as arestas lógicas, precisamos exercitar cada comando condicional como True e False.

• **verifica_numero(n)**

- n = 6 (Cobre: n > 0 [V], n % 2 == 0 [V], n % 3 == 0 [V])
- n = 10 (Cobre: n > 0 [V], n % 2 == 0 [V], n % 3 == 0 [F], n % 5 == 0 [V])
- n = 2 (Cobre: n > 0 [V], n % 2 == 0 [V], n % 3 == 0 [F], n % 5 == 0 [F])
- n = 9 (Cobre: n > 0 [V], n % 2 == 0 [F], n % 3 == 0 [V])
- n = 25 (Cobre: n > 0 [V], n % 2 == 0 [F], n % 3 == 0 [F], n % 5 == 0 [V])
- n = 7 (Cobre: n > 0 [V], n % 2 == 0 [F], n % 3 == 0 [F], n % 5 == 0 [F])
- n = -1 (Cobre: n > 0 [F], n < 0 [V])
- n = 0 (Cobre: n > 0 [F], n < 0 [F])

• **compara_numeros(a, b)**

- a = 5, b = 5 (Cobre: a == b [V])
- a = 10, b = 5 (Cobre: a == b [F], a > b [V], a % b == 0 [V])
- a = 10, b = 3 (Cobre: a == b [F], a > b [V], a % b == 0 [F])
- a = 5, b = 10 (Cobre: a == b [F], a > b [F], b % a == 0 [V])

- $a = 3, b = 10$ (Cobre: $a == b$ [F], $a > b$ [F], $b \% a == 0$ [F])
- **analisa_string(s)**
 - $s = ""$ (Cobre: $\text{len}(s) == 0$ [V])
 - $s = "ABC"$ (Cobre: $\text{len}(s) == 0$ [F], $\text{isalpha}()$ [V], $\text{isupper}()$ [V])
 - $s = "abc"$ (Cobre: $\text{len}(s) == 0$ [F], $\text{isalpha}()$ [V], $\text{isupper}()$ [F], $\text{islower}()$ [V])
 - $s = "Abc"$ (Cobre: $\text{len}(s) == 0$ [F], $\text{isalpha}()$ [V], $\text{isupper}()$ [F], $\text{islower}()$ [F])
 - $s = "123"$ (Cobre: $\text{len}(s) == 0$ [F], $\text{isalpha}()$ [F], $\text{isdigit}()$ [V])
 - $s = "A1"$ (Cobre: $\text{len}(s) == 0$ [F], $\text{isalpha}()$ [F], $\text{isdigit}()$ [F], $\text{any}(\text{isalnum})$ [V])
 - $s = "@\#\$\"$ (Cobre: $\text{len}(s) == 0$ [F], $\text{isalpha}()$ [F], $\text{isdigit}()$ [F], $\text{any}(\text{isalnum})$ [F])
- **processa_logico(condicao)**
 - $\text{condicao} = \text{True}$ (Cobre: condicao [V], laço for executado, $i \% 2 == 0$ [V e F])
 - $\text{condicao} = \text{False}$, input 1 = 's', input 2 = '5' (Cobre: condicao [F], $\text{lower}() == 's'$ [V], > 0 [V])
 - $\text{condicao} = \text{False}$, input 1 = 's', input 2 = '-2' (Cobre: condicao [F], $\text{lower}() == 's'$ [V], > 0 [F])
 - $\text{condicao} = \text{False}$, input 1 = 'n' (Cobre: condicao [F], $\text{lower}() == 's'$ [F])

b. GFC e Complexidade Ciclomática ($V(G) = P + 1$) A métrica de McCabe é calculada com base no número de nós predicativos (P) mais 1.

1. **verifica_numero**
 - *Nós predicativos (7):* $n > 0$, $n \% 2 == 0$, $n \% 3 == 0$ (par), $n \% 5 == 0$ (par), $n \% 3 == 0$ (ímpar), $n \% 5 == 0$ (ímpar), $n < 0$.
 - *Complexidade Ciclomática: $V(G) = 8$*
2. **compara_numeros**
 - *Nós predicativos (4):* $a == b$, $a > b$, $a \% b == 0$, $b \% a == 0$.
 - *Complexidade Ciclomática: $V(G) = 5$*
3. **analisa_string**
 - *Nós predicativos (6):* $\text{len}(s) == 0$, $s.\text{isalpha}()$, $s.\text{isupper}()$, $s.\text{islower}()$, $s.\text{isdigit}()$, $\text{any}(c.\text{isalnum}() \text{ for } c \text{ in } s)$.
 - *Complexidade Ciclomática: $V(G) = 7$*
4. **processa_logico**
 - *Nós predicativos (5):* if condicao, for i in range(3) (condição de parada do laço iterativo), $i \% 2 == 0$, $\text{input}() == 's'$, $\text{int}(\text{input}()) > 0$.
 - *Complexidade Ciclomática: $V(G) = 6$*
 -

4. (0,5) Considerando o arquivo lab2.py ou a aplicação que vocês apresentaram nos seminários a. Determine o domínio de entrada e de saída de cada uma das funções

b. Crie um conjunto de testes para pelo menos 4 funções com base em algum critério funcional visto em aula.

Resposta:

a. Determinação do domínio de entrada e saída

1. verifica_numero

- *Entrada:* $\mathbb{Z}$ (Conjunto dos números inteiros).
- *Saída:* String categórica identificando a divisibilidade e o sinal do número (9 retornos textuais possíveis).

2. compara_numeros

- *Entrada:* $\mathbb{Z} \times \mathbb{Z}$ (Par ordenado de números inteiros).
- *Saída:* String identificando a relação de grandeza e multiplicidade.

3. analisa_string

- *Entrada:* Σ^* (Qualquer cadeia de caracteres finita, incluindo string vazia).
- *Saída:* String categórica descrevendo o tipo de conteúdo léxico da cadeia.

4. processa_logico

- *Entrada:* Booleano (True ou False) nos parâmetros; interações de sistema do tipo String e Inteiro capturadas em tempo de execução via input().
- *Saída:* String indicando o processamento efetuado e mensagens exibidas no console (efeitos colaterais).

b. Conjunto de testes (Critério: Particionamento em Classes de Equivalência)

• **verifica_numero(n)**

- *Classe 1 (Positivo Par Múltiplo de 3):* $n = 12$
- *Classe 2 (Positivo Ímpar Múltiplo de 5):* $n = 25$
- *Classe 3 (Negativo):* $n = -42$
- *Classe 4 (Zero):* $n = 0$

• **compara_numeros(a, b)**

- *Classe 1 (Iguais):* $a = 7, b = 7$
- *Classe 2 (A maior e divisível por B):* $a = 20, b = 4$
- *Classe 3 (B maior e não divisível por A):* $a = 3, b = 8$

• **analisa_string(s)**

- *Classe 1 (Caracteres Maiúsculos):* $s = \text{"PYTHON"}$
- *Classe 2 (Alfanumérico misto):* $s = \text{"Teste123"}$
- *Classe 3 (Sem alfanuméricos):* $s = \text{"!!!@@@@"}$
- *Classe 4 (Vazia):* $s = \text{"}"}$

• **processa_logico(condicao)**

- *Classe 1 (True):* $\text{condicao} = \text{True}$
- *Classe 2 (False + Confirmação Positiva):* $\text{condicao} = \text{False}$, Usuário digita 's', Usuário digita '10'
- *Classe 3 (False + Aborto):* $\text{condicao} = \text{False}$, Usuário digita 'n'